

Arreglos Sistólicos y Arquitecturas de Dominio Específico

Elly Cortez

*Escuela de Ingeniería Electrónica
Tecnológico de Costa Rica
Cartago, Costa Rica
ecortez@estudiantec.cr*

Juan P. Elizondo

*Escuela de Ingeniería Electrónica
Tecnológico de Costa Rica
Cartago, Costa Rica
juelizondo@estudiantec.cr*

Luis D. García

*Escuela de Ingeniería Electrónica
Tecnológico de Costa Rica
Cartago, Costa Rica
guisogarcia1010@estudiantec.cr*

Yonaikel F. Tencio

*Escuela de Ingeniería Electrónica
Tecnológico de Costa Rica
Cartago, Costa Rica
tencioyonaikel@estudiantec.cr*

I. INTRODUCCIÓN

II. FUNDAMENTOS TEÓRICOS

...

III. NÚCLEOS TENSORIALES

Ante el rápido avance de la inteligencia artificial y del aprendizaje profundo, las unidades de cómputo tradicionales (como los procesadores de flujo convencionales), se volvieron insuficientes para satisfacer la carga computacional requerida por los tensores de gran tamaño (estructuras de datos multidimensionales).

Debido a esta limitante, surgió la necesidad de integrar estructuras de aceleración heterogénea especializadas en operadores específicos. Ante esto, la empresa NVIDIA introdujo por primera vez una unidad de procesamiento de tensores dedicada, conocida como Tensor Core (Núcleo Tensorial). Su propósito era subsanar las deficiencias de cálculo matricial de sus GPUs convencionales, por medio de la aceleración por hardware de precisión mixta, con el objetivo de reducir los tiempos de ejecución al procesar volúmenes masivos de datos.

III-A. El Arreglo Sistólico como el núcleo del Tensor Core

Para comprender cómo los núcleos tensoriales logran esta eficiencia disruptiva frente a las unidades tradicionales de la GPU, es crucial vincularlos con el concepto de los arreglos sistólicos (systolic arrays). Un sistema de cómputo tradicional se ve drásticamente limitado por los constantes accesos y transferencias hacia la memoria principal. En contraste, un núcleo tensorial moderno no reinventa las matemáticas del álgebra lineal, sino que adopta la estructura paralela de un arreglo sistólico bidimensional como su propio núcleo de procesamiento computacional.

La arquitectura del núcleo tensorial aprovecha precisamente la ventaja competitiva de los arreglos sistólicos: una malla o red bidimensional de Unidades de Procesamiento (PE, Processing Units) interconectadas directamente a nivel de hardware mediante una segmentación (pipeline) balanceada.

III-B. La operación MMA

Los Tensor Cores están diseñados específicamente para resolver la ecuación de multiplicación y acumulación de matrices (MMA, Matrix Multiply-Accumulate) [1], según muestra la ecuación 1:

$$D = A \times B + C \quad (1)$$

En este proceso, A y B son las matrices que se multiplican entre sí. Las cuales suelen estar en formatos de baja precisión como FP16 (16 bits de punto flotante) o INT8 (8 bits enteros), lo que permite una mayor velocidad de procesamiento.

La matriz C es la matriz de acumulación, generalmente manejada con una precisión mayor como FP32 (32 bits de punto flotante) para evitar la pérdida de información y mitigar los errores de redondeo.

Por último, el resultado final de la operación se entrega en la matriz D , también de alta precisión.

III-C. Flujo de ejecución en el Hardware

Las matrices de los modelos de aprendizaje profundo, son muy grandes, por lo que un núcleo tensorial no puede procesarlas de golpe. En su lugar, se puede aplicar una estrategia jerárquica de división y ejecución, basada en el flujo de datos de un arreglo sistólico [1].

III-C1. Segmentación por bloques: Las matrices de gran tamaño (A , B y C) se fragmentan en subtarefas de dimensiones fijas que coinciden exactamente con el tamaño físico del arreglo de hardware.

III-C2. Distribución escalonada de datos: La manera en la que los datos son introducidos al núcleo tensorial, no es en forma de un bloque completo, sino que se introducen de manera síncrona y escalonada, por filas y columnas. Lo anterior implica que cada elemento de la matriz entra al circuito con un retraso de un ciclo de reloj respecto a su fila o columna vecina, lo que crea una onda de datos coordinados.

III-C3. El flujo sistólico y el reúso de datos: Aquí es donde entra en juego la magia de los arreglos sistólicos. Una vez que los datos se encuentran dentro de la cuadrícula bidimensional (de PEs), estos fluyen siguiendo el ritmo de un "pulso cardíaco". De esta forma, cada unidad PE recibe un elemento de la matriz A por la izquierda y un elemento de la matriz B desde arriba, realizando la multiplicación de los dos elementos individuales y sumándoles un valor acumulado intermedio; inmediatamente después, la PE pasa el elemento de A y B de la misma forma que como se mencionó al inicio.

III-C4. Acumulación y resultado final: Cada unidad PE aprovecha su registro de acumulación para ir almacenando el resultado de las multiplicaciones parciales. En la práctica, se suele "precargar-la matriz C antes de la entrada de los datos provenientes de las matrices A y B , de esta manera tras sumar cada valor de la matriz C y los valores acumulados de las multiplicaciones parciales, el resultado final de la matriz D se encuentra consolidado.

III-D. El problema de la subutilización

Un problema en la aplicación de los núcleos tensoriales, aparece al momento de considerar todo el sistema arquitectónico que los rodea, ya que esto suma limitaciones al sistema que genera una severa subutilización de los núcleos tensoriales [2]. Lo anterior se puede explicar por tres razones principalmente.

- **La asimetría de la jerarquía de memoria:** Las operaciones de multiplicación de matrices generales (GEMM), requieren un flujo constante y masivo de datos. Mientras que los *tensor cores* pueden procesar los datos casi instantáneamente a nivel de registros, el tiempo que toma mover los datos desde la memoria global hasta la memoria compartida, y luego a los registros de los hilos, es órdenes de magnitud más lento.
- **Dependencias de datos e instrucciones (Latencia del Pipeline):** Las instrucciones de "soporte"provenientes del software de la GPU, llegan a saturar las unidades de ejecución, lo que provoca que la emisión de instrucciones hacia el *tensor core* sea intermitente.
- **Acoplamiento rígido de recursos:** Como los núcleos tensoriales están asignados fijamente a un número específico de unidades de procesamiento, en caso de que el grupo de hilos se llegue a retrasar, ya sea buscando datos en memoria o resolviendo una bifurcación de código, el *tensor core* queda congelado, desperdiciando ciclos de reloj valiosos.

III-E. La dimensión de ejecución y el rol de los warps

El *Warp* es el bloque básico de ejecución en el modelo SIMT de las GPU, el cual consiste en un grupo de hilos que ejecutan la misma instrucción de forma simultánea, sobre diferentes datos [2].

Bajo este nivel de abstracción, el programador de GPU escribe el código pensando en hilos individuales; no obstante, un solo hilo de la GPU no puede controlar un núcleo tensorial. Además, la operación matemática MMA se expone al sistema como una instrucción intrínseca a nivel de Warp. Al final,

ningún hilo posee la matriz completa; cada hilo guarda en sus registros privados una pequeña porción de las matrices de entrada A y B .

Una solución a la subutilización de los núcleos tensoriales, es que múltiples Warps compartan el mismo *tensor core* de manera dinámica, esto mediante un planificador especializado a nivel de arquitectura. De esta forma, mientras un Warp se encuentra en espera, el hardware conmuta de inmediato el contexto para que otro Warp totalmente independiente tome el control del núcleo tensorial e inicie su propio cálculo [2].

IV. TPU DE GOOGLE

V. OPTIMIZACIÓN DE ARREGLOS MAC

REFERENCIAS

- [1] Z. Feng, H. Zhang, A. Li, L. Xing, D. Wu, and Y. Liu, "A tensor core architecture for large-scale matrix multiplication operations on gpgpu processor," in *2025 4th International Symposium on Semiconductor and Electronic Technology (ISSET)*. IEEE, 2025, pp. 246–249.
- [2] A. Nada, G. M. Sarda, and E. Lenormand, "Cooperative warp execution in tensor core for risc-v gpgpu," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2025, pp. 1424–1436.